МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Алгоритмы и структуры данных»

Тема: Реализация и исследование кольцевого буфера

Студент гр. 5387 _____ Степанов Л. А.

Преподаватель _____ Суворов А. А.

Санкт-Петербург
2026

Оглавление

Цель работы 3

Основные теоретические положения 5

Выполнение работы 6

Pipeline данных 9

Исследование 11

Вывод 17

ПРИЛОЖЕНИЕ А 18

Исходный код программы 18

ПРИЛОЖЕНИЕ Б 29

Результаты тестирования 29

Цель работы

Реализовать класс CircularDeque - двустороннюю очередь (деку) на кольцевом буфере фиксированной длины.

Задание

Требуется реализовать кольцевой буфер для целых чисел на основе статического массива длины N. Реализация должна представлять из себя класс CircularDeque. В поле предварительного ответа предложены сигнатуры класса для python и c++.

В конструкторе класс должен принимать количество элементов N в массиве, а также значение bool push_force, отражающее, будут ли при переполнении буфера элементы перезаписывать старые.

Класс должен поддерживать следующие методы (каждый за O(1) в худшем случае или амортизированно):

- void push_front(x) - добавить элемент x в начало деки;
- void push_back(x) - добавить элемент x в конец деки;

Если дека заполнена, а push_force указан как false, при попытке добавить элементы в деку должно выбрасываться исключение; если же push_force указан как true, то при добавлении элемента в конец деки он должен записаться поверх элемента с начала деки, а при добавлении элемента в начало - поверх элемента с конца. При этом начало/конец деки должно сместиться на последующий/предыдущий элемент.

Пример:

Пусть дека вместимости 3 заполнена: [1, 2, 3], front = 1, back = 3, push_force = true. При выполнении push_back(4) элемент 4 запишется поверх начала деки - элемента 1, а само начало деки сдвинется на следующий элемент - 2: [4, 2, 3], front = 2, back = 4.

- int pop_front() - удалить и вернуть элемент из начала деки;
- int pop_back() - удалить и вернуть элемент из конца деки;

- int front() - вернуть (не удаляя сам элемент) значение первого элемента в деке;
- int back() - вернуть значение последнего элемента;
- int size() - вернуть текущее число элементов в деке;
- bool empty() - проверить деку на пустоту;
- bool full() - проверить деку на заполненность (если массив полон).
- void resize(new_capacity) - изменить размер статического массива и перезаписать элементы в массив нового размера. При перезаписи необходимо перемещать элементы в правильном порядке - от начала деки к её концу. Если не все элементы старого массива помещаются в новый, необходимо оставить лишь new_capacity первых (с начала деки) элементов массива.

Если выполнить какую-либо операцию невозможно, необходимо выбросить исключение.

Основные теоретические положения

Двусторонняя очередь (дека) - контейнер, который совмещает свойства стека и очереди. Добавлять и удалять элементы можно с любого конца, и каждая такая операция выполняется за константное время O(1).

Чтобы получить такую скорость, деку удобно строить на циклическом массиве - буфере постоянного размера, замкнутом в кольцо через остаток от деления.

Сами данные при этом никуда не двигаются: хранятся только два индекса - head для начала и tail для конца, а любая вставка или удаление сводится к пересчёту этих индексов. Элементы остаются на своих местах, поэтому push_front, push_back, pop_front и pop_back работают за фиксированное время, сколько бы элементов ни лежало в буфере.

Заодно это экономит память: массив выделяется один раз и дальше просто используется по кругу.

Выполнение работы

В основе решения лежит вектор std::vector<int> фиксированной вместимости, индекс головы head и счётчик элементов count. Отдельный индекс хвоста не хранится: позицию за последним элементом считаем как (head + count) по модулю вместимости. Ниже описан класс CircularDeque.

Описание класса CircularDeque

Класс CircularDeque - это кольцевая дека на массиве фиксированного размера. Он умеет работать в режиме, когда при заполнении буфера старые элементы вытесняются новыми.

Поля класса:

- std::vector<int> arr - массив-хранилище фиксированной вместимости cap.
- size_t cap - предельное число элементов, помещающихся в буфер.
- size_t count - текущее число элементов в деке.
- size_t head - позиция первого (головного) элемента в кольцевом буфере.
- bool force - задаёт поведение при переполнении: true разрешает затирать самые старые элементы, false приводит к исключению при попытке вставки в полный буфер.

Методы класса:

- CircularDeque(size_t N, bool push_force = false) - конструктор: N задаёт вместимость (при N == 0 выбрасывается std::invalid_argument), push_force устанавливает режим вытеснения.
- ensure_full() - вспомогательный: если буфер полон и force = false, выбрасывает std::overflow_error.
- ensure_not_empty() - вспомогательный: выбрасывает std::underflow_error, когда дека пуста.
- next_index(x), prev_index(x) - вспомогательные: вычисляют (x + 1) % cap и (x + cap − 1) % cap для перемещения по кольцу; back_index() и

free_back_index() возвращают индекс последнего элемента и первой свободной ячейки за ним.

- push_back(int x) - кладёт элемент x в хвост деки.
- push_front(int x) - кладёт элемент x в голову деки.
- int pop_front() - снимает и возвращает элемент с головы.
- int pop_back() - снимает и возвращает элемент с хвоста.
- int& front() - читает головной элемент, не удаляя его.
- int& back() - читает хвостовой элемент, не удаляя его.
- int size() const - возвращает текущее число элементов.
- bool empty() const - признак пустой деки.
- bool full() const - признак полностью занятого буфера.
- void resize(size_t new_cap) - меняет вместимость и переносит содержимое в новый массив с сохранением порядка от головы к хвосту.

Детальное описание методов:

void push_front(int x)

Вставляет элемент в голову деки. Сначала вызывается ensure_full(): если буфер полон и force = false, бросается std::overflow_error с текстом «Index out of range!». Индекс головы сдвигается назад по кольцу - idx = prev_index(head), в arr[idx] пишется x, после чего head становится равным idx. Если место ещё есть (count < cap), счётчик count растёт. При force = true и полном буфере запись в prev_index(head) попадает на крайний элемент хвоста, то есть новый элемент вытесняет конец деки, а count так и остаётся равным cap.

void push_back(int x)

Так же добавляет элемент в хвост. После ensure_full(): если буфер полон (cap == count, а это возможно только при force = true), новый элемент пишется поверх головы - arr[head] = x, а head двигается вперёд next_index(head); голова деки вытесняется, count остаётся равным cap. Если место есть, x пишется в первую свободную ячейку arr[free_back_index()], и count растёт.

int pop_front()

Убирает элемент с головы. Сначала ensure_not_empty(): если дека пуста, бросается std::underflow_error с текстом «Deque is empty». Иначе запоминаем значение arr[head], двигаем head вперёд next_index(head), уменьшаем count и возвращаем сохранённое значение.

int pop_back()

Убирает элемент с хвоста. После ensure_not_empty() считаем индекс последнего элемента idx = back_index() = (head + count − 1) % cap, читаем arr[idx], уменьшаем count и возвращаем значение.

int& front()

Возвращает ссылку на головной элемент, не удаляя его. На пустой деке ensure_not_empty() бросает std::underflow_error. Возврат по ссылке (int&) даёт возможность и прочитать, и изменить головной элемент.

int& back()

Возвращает ссылку на хвостовой элемент, не удаляя его; его позиция считается как back_index() = (head + count − 1) % cap. На пустой деке снова бросается std::underflow_error.

void resize(size_t new_cap)

Перестраивает буфер под вместимость new_cap (при new_cap == 0 - std::invalid_argument с текстом «new_cap must be > 0»). Число переносимых элементов new_count = min(count, new_cap). Элементы копируются по порядку от головы к хвосту: new_arr[i] = arr[(head + i) % cap]. В конце массив заменяется новым, cap = new_cap, head = 0, count = new_count. Если новая вместимость меньше текущего размера, остаются только первые new_cap элементов.

Чтобы проверить, что всё работает правильно, написан набор тестов - на обычные операции и на граничные случаи вместе со всеми исключениями. Полный код программы приведён в приложении А, тексты тестов - в приложении Б, а результаты их выполнения сведены в таблицу 1 (приложение Б).

Pipeline данных

Проследим по шагам за состоянием деки CircularDeque(4) с push_force = true. Отдельного индекса хвоста нет: где голова, задаёт head, а сколько элементов - count.

Начальное состояние

arr: [_, _, _, _]

head: 0, count: 0

1. dq.push_back(0);

Буфер не полон. Значение 0 кладётся в свободную ячейку arr[free_back_index() = (head + count) % 4 = (0 + 0) % 4 = 0]; счётчик count становится равным 1.

Состояние: arr: [0, _, _, _]; head: 0, count: 1

2. dq.push_back(1);

Значение 1 записывается в arr[free_back_index() = (0 + 1) % 4 = 1]; count → 2.

Состояние: arr: [0, 1, _, _]; head: 0, count: 2

3. dq.push_front(2);

Место ещё есть. head уходит назад: prev_index(0) = (0 + 4 − 1) % 4 = 3; в arr[3] пишется 2, head становится равным 3; count → 3.

Состояние: arr: [0, 1, _, 2]; head: 3, count: 3 (логически в деке: 2, 0, 1)

4. dq.pop_back();

Индекс последнего элемента: back_index() = (head + count − 1) % 4 = (3 + 3 − 1) % 4 = 1; возвращается arr[1] = 1; count → 2. Индекс head не меняется, ячейка физически не очищается.

Состояние: arr: [0, 1, _, 2]; head: 3, count: 2 (логически в деке: 2, 0)

5. dq.push_back(3);

Свободная ячейка: free_back_index() = (head + count) % 4 = (3 + 2) % 4 = 1; значение 3 пишется в arr[1]; count → 3.

Состояние: arr: [0, 3, _, 2]; head: 3, count: 3 (логически в деке: 2, 0, 3)

6. dq.push_back(4);

free_back_index() = (3 + 3) % 4 = 2; значение 4 пишется в arr[2]; count → 4 - буфер заполнен.

Состояние: arr: [0, 3, 4, 2]; head: 3, count: 4 (логически в деке: 2, 0, 3, 4)

7. dq.push_back(5); (с push_force = true)

Буфер полон (count == cap), но force = true, поэтому ensure_full() не выбрасывает исключение. Так как cap == count, значение 5 пишется прямо поверх головы: arr[head = 3] = 5, после чего head продвигается вперёд next_index(3) = (3 + 1) % 4 = 0. Прежний головной элемент (2) вытеснен, count остаётся равным 4.

Состояние: arr: [0, 3, 4, 5]; head: 0, count: 4 (логически в деке: 0, 3, 4, 5)

8. dq.pop_front();

Возвращается arr[head = 0] = 0; head идёт вперёд next_index(0) = (0 + 1) % 4 = 1; count → 3.

Итоговое состояние: arr: [0, 3, 4, 5]; head: 1, count: 3 (логически в деке: 3, 4, 5; front = 3, back = 5)

На этой цепочке хорошо видно, как работает кольцевая дека: буфер используется по кругу, добавлять и убирать элементы с обоих концов дёшево, а когда места не остаётся, включается вытеснение.

Исследование

Чтобы оценить сложность операций по времени, был проведён нагрузочный тест. Проверялись такие гипотезы о времени работы основных операций:

- константное (амортизированное) время O(1) - вставка (push_front, push_back);
- константное время O(1) - извлечение (pop_front, pop_back) и доступ к крайним элементам (front, back);
- линейное время O(n) - перестроение буфера (resize).

Каждую операцию замеряли отдельно на буферах разного размера - от 1000 до 10 000 000 элементов. На каждый размер делалось несколько повторов, а за результат бралась медиана. Для сравнения на каждый график нанесена теоретическая кривая: горизонтальная линия O(1) для константных операций и прямая O(n) для resize. Код нагрузочного теста и построения графиков приведён в приложении В.

Исходные данные, используемые при исследовании:

- тип элементов - целые числа (int); буфер создавался в режиме push_force = false;
- размеры буфера N - от 1000 до 10 000 000 элементов с шагом в 1,5 раза (около 24 точек);
- число прогревочных прогонов WARMUP = 1, число измеряемых повторов REPEAT = 9; результатом служила медиана;
- для push_back и push_front выполнялось N вставок в изначально пустой буфер;
- для pop_back, pop_front, front и back буфер предварительно заполнялся N элементами, после чего выполнялось N операций;
- для resize буфер заполнялся N элементами и однократно расширялся до 2N;
- измеряемая величина - среднее время одной операции в наносекундах (для resize - время одного вызова).

Ниже на рисунках 1-7 приведены результаты замеров для каждой операции в отдельности.

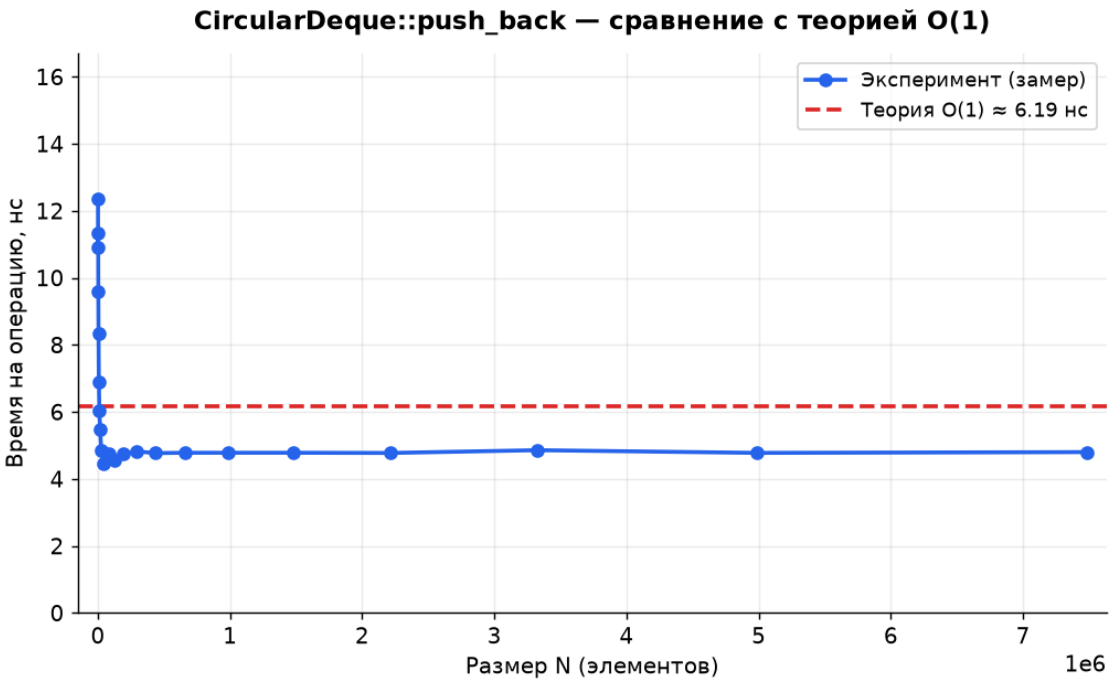


Рисунок 1 - Производительность операции push_back

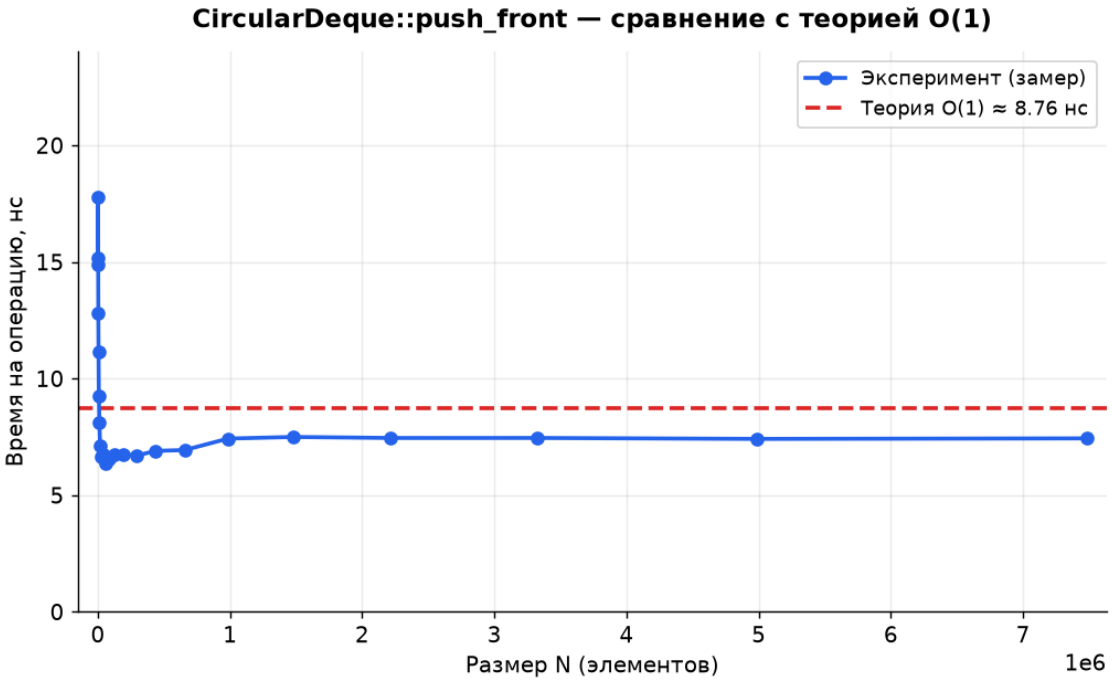


Рисунок 2 - Производительность операции push_front

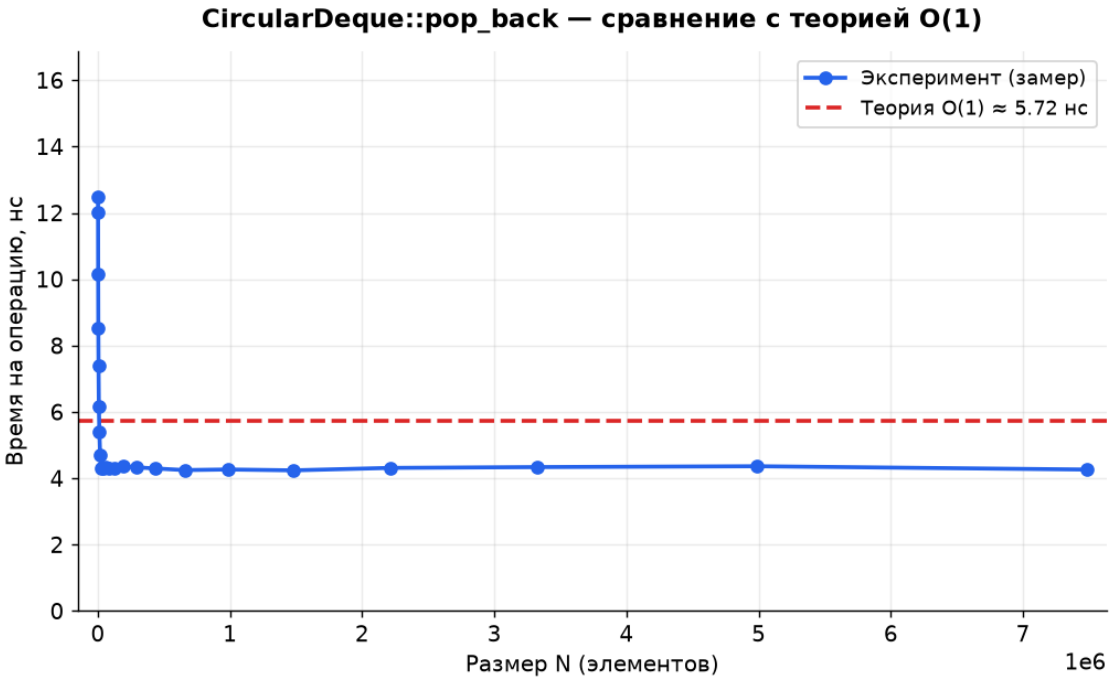


Рисунок 3 - Производительность операции pop_back

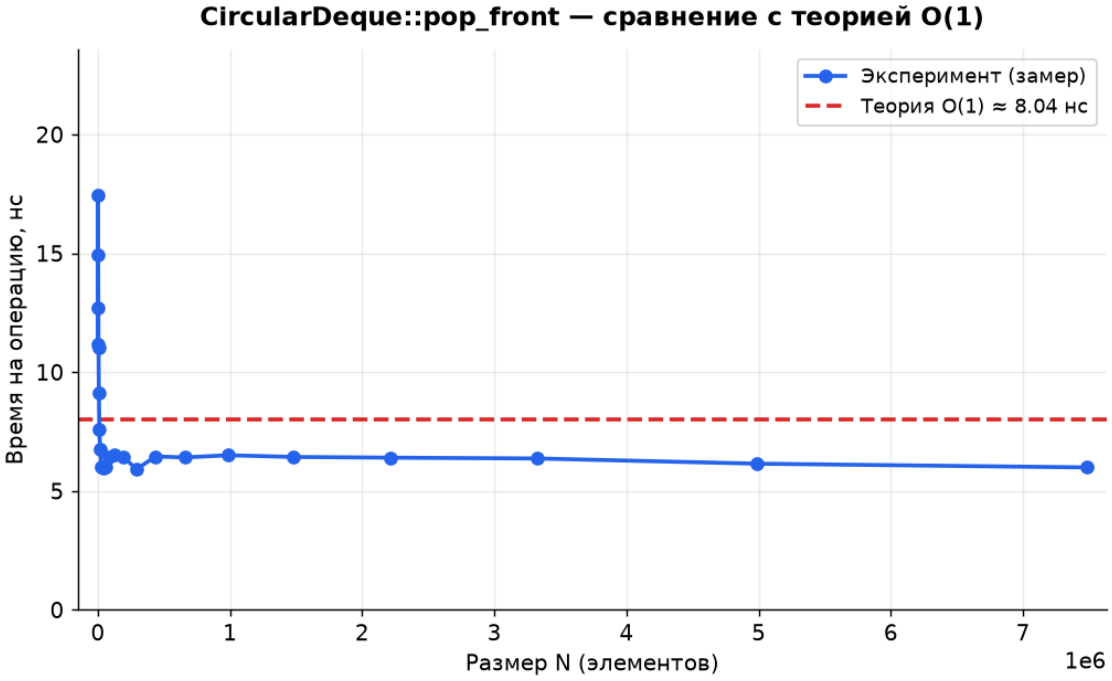


Рисунок 4 - Производительность операции pop_front

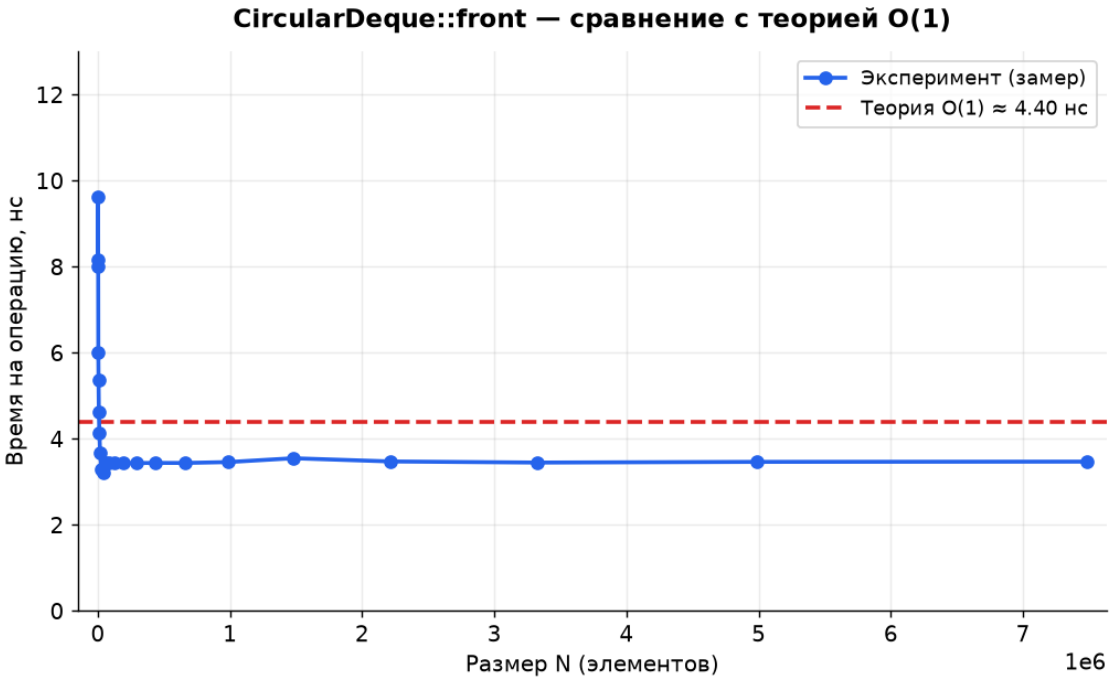


Рисунок 5 - Производительность операции front

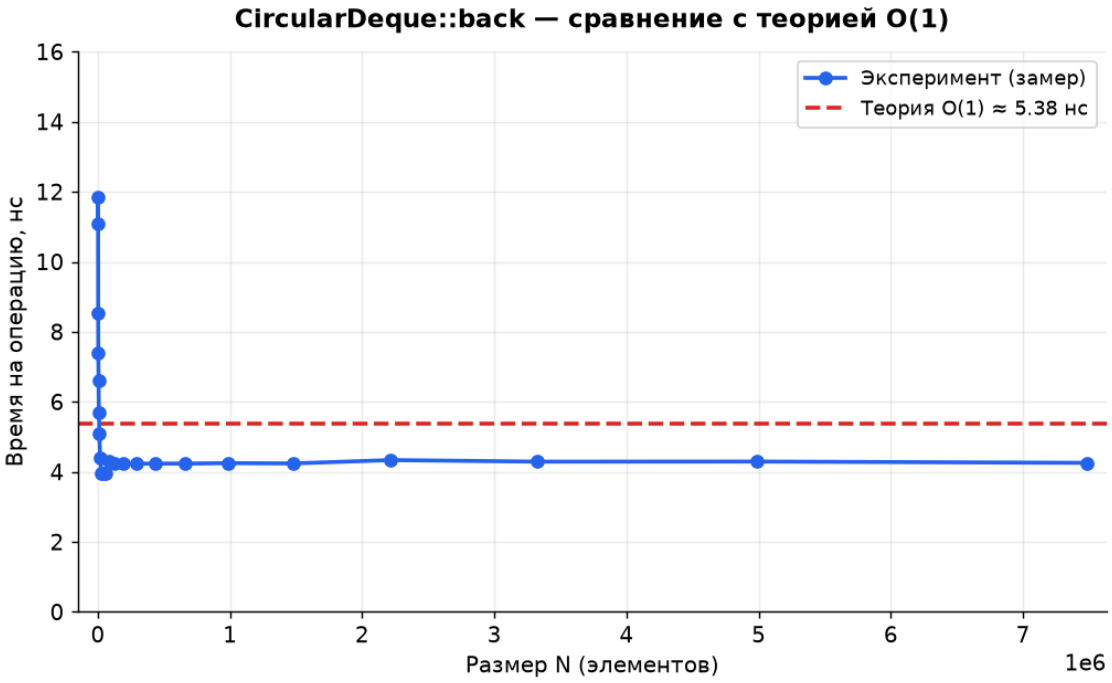


Рисунок 6 - Производительность операции back

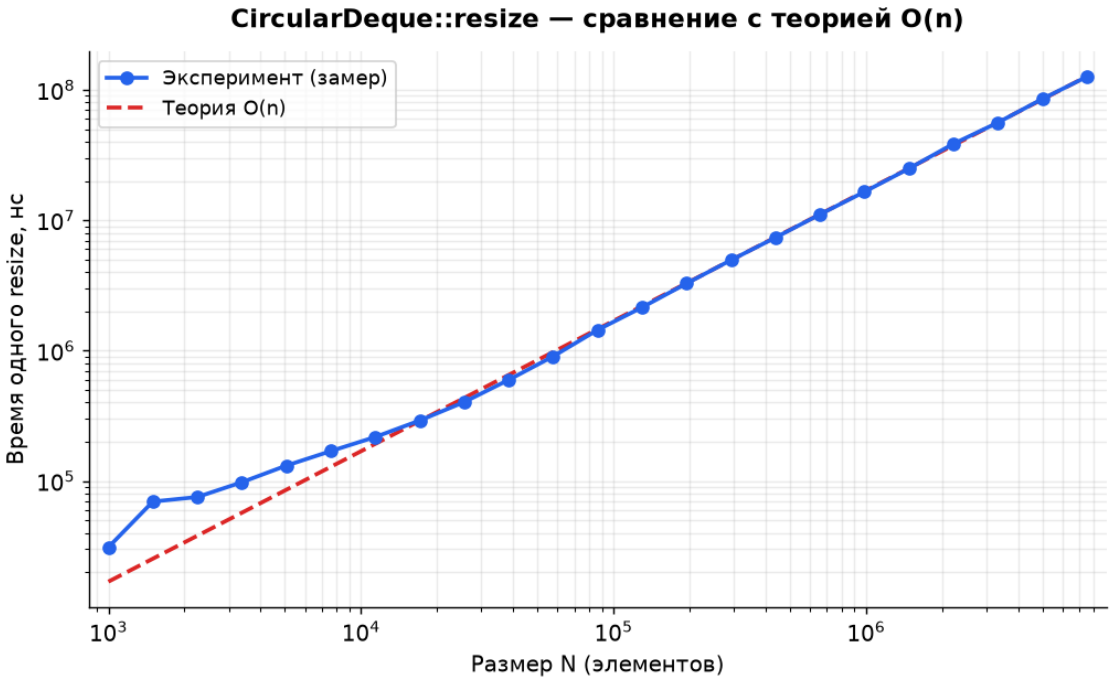


Рисунок 7 - Время работы операции resize (сравнение с O(n))

Основные наблюдения

- Время у push_back, push_front, pop_back, pop_front, front и back почти не меняется: замеры держатся в узкой полосе, примерно 4–9 нс, хотя размер буфера меняется на четыре порядка - от 1000 до 10 000 000 элементов. Это и есть амортизированное O(1) (см. рисунки 1–6).
- Совпадение с теорией O(1): кривые константных операций идут почти горизонтально, рядом с теоретической линией, и не растут линейно.
- Всплеск на маленьких N: первые точки доходят до 10–18 нс - сказывается прогрев кэша и планировщик ОС. С ростом N время выравнивается, то есть от размера буфера алгоритм не зависит.

Сложность метода resize

Метод resize копирует все элементы в новый массив: цикл делает new_count (не больше N) присваиваний, поэтому по времени это O(n), а по памяти O(new_cap). В отличие от остальных операций, resize не константный. Рисунок 7 это подтверждает: в двойных логарифмических координатах кривая почти ложится на теоретическую прямую O(n) - время одного вызова resize растёт пропорционально размеру буфера (примерно от 30 000 нс при N = 1000 до 100 000 000 нс при N около 7 000 000).

Из замеров видно, что CircularDeque даёт:

- амортизированное O(1) на операциях вставки, извлечения и доступа;
- почти одинаковое время работы независимо от объёма данных;
- предсказуемое время - без резких скачков на больших N;
- линейное O(n) только на resize - это ожидаемо, меньше для копирующего перестроения буфера не сделать.

В целом реализация ведёт себя так, как и предсказывает теория.

Вывод

В работе разобрано, как устроен и как работает кольцевой буфер (дека) на статическом массиве: вытеснение старых элементов по флагу push_force, вставка и удаление с обоих концов и изменение вместимости без потери данных.

На C++ написан класс CircularDeque, который закрывает все требования задания. Константное время операций держится на модульной арифметике - за неё отвечают вспомогательные методы next_index, prev_index, back_index и free_back_index, которые считают адреса по кольцу. Реализованы все нужные операции: push_front, push_back, pop_front, pop_back, front, back, size, empty, full и resize, а также аккуратно обработаны крайние случаи - пустой и полностью занятый буфер.

Некорректные операции отсекаются обычными исключениями C++ (throw): std::invalid_argument при недопустимой вместимости, std::overflow_error при переполнении и std::underflow_error при обращении к пустой деке. Режим push_force сделан через перезапись противоположного края в момент переполнения: при push_back затирается голова, при push_front - хвост.

ПРИЛОЖЕНИЕ А

Исходный код программы

Файл src/main.cpp

```
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>

#include "modules/circular_deque.h"

using namespace std;

static void print_help() {
    cout << "Команды:\n"
         << "  push_front X   — добавить X в начало\n"
         << "  push_back X    — добавить X в конец\n"
         << "  pop_front      — извлечь элемент из начала\n"
         << "  pop_back       — извлечь элемент из конца\n"
         << "  front          — первый элемент\n"
         << "  back           — последний элемент\n"
         << "  size           — количество элементов\n"
         << "  empty          — пуст ли дек\n"
         << "  full           — заполнен ли дек\n"
         << "  resize N       — изменить вместимость\n"
         << "  help           — эта справка\n"
         << "  exit           — выход\n";
}

int main() {
    size_t n;
    int force;
    cout << "Введите вместимость дека N и режим push_force (0/1): ";
    if (!(cin >> n >> force)) {
        cerr << "Ошибка: ожидались два числа" << endl;
        return 1;
    }

    CircularDeque* dq;
    try {
        dq = new CircularDeque(n, force != 0);
    } catch (const exception& e) {
        cerr << "Ошибка: " << e.what() << endl;
        return 1;
    }

    print_help();

    string line;
    getline(cin, line);
    while (cout << "> " && getline(cin, line)) {
        istringstream in(line);
        string cmd;
        if (!(in >> cmd)) continue;

        try {
            if (cmd == "push_front" || cmd == "push_back" || cmd == "resize") {
                long long x;
                if (!(in >> x)) {
                    cout << "Ошибка: команде " << cmd << " нужен аргумент" << endl;
                    continue;
                }
                if (cmd == "push_front") dq->push_front(static_cast<int>(x));
                else if (cmd == "push_back") dq->push_back(static_cast<int>(x));
                else {
                    if (x <= 0) throw invalid_argument("new_cap must be > 0");
                    dq->resize(static_cast<size_t>(x));
```

```
                }
                cout << "OK" << endl;
            } else if (cmd == "pop_front") {
                cout << dq->pop_front() << endl;
            } else if (cmd == "pop_back") {
                cout << dq->pop_back() << endl;
            } else if (cmd == "front") {
                cout << dq->front() << endl;
            } else if (cmd == "back") {
                cout << dq->back() << endl;
            } else if (cmd == "size") {
                cout << dq->size() << endl;
            } else if (cmd == "empty") {
                cout << (dq->empty() ? "true" : "false") << endl;
            } else if (cmd == "full") {
                cout << (dq->full() ? "true" : "false") << endl;
            } else if (cmd == "help") {
                print_help();
            } else if (cmd == "exit" || cmd == "quit") {
                break;
            } else {
                cout << "Неизвестная команда: " << cmd << " (help — список команд)"
<< endl;
            }
        } catch (const exception& e) {
            cout << "Ошибка: " << e.what() << endl;
        }
    }

    delete dq;
    return 0;
}
```

Файл src/modules/circular_deque.h

```
#pragma once

#include <iostream>
#include <vector>
#include <stdexcept>

using namespace std;

class CircularDeque
{
private:
    vector<int> arr;
    size_t cap;
    size_t head;
    size_t count;
    bool force;

    size_t back_index() const {
        return (head + count - 1) % cap;
    }

    size_t free_back_index() const {
        return (head + count) % cap;
    }

    void ensure_full() const{
        if(this->count == this->cap && this->force == false){
            throw overflow_error("Index out of range!");
        }
    }

    void ensure_not_empty() const{
        if(this->count == 0){
            throw underflow_error("Deque is empty");
        }
    }
```

```
    size_t next_index(size_t x) const { return (x + 1) % cap; }
    size_t prev_index(size_t x) const { return (x + cap - 1) % cap; }

public:
    explicit CircularDeque(size_t N, bool push_force = false){
        if(N == 0) throw invalid_argument("N must be N>0");
        this->cap = N;
        this->head = 0;
        this->count = 0;
        this->force = push_force;
        this->arr.resize(N);
    }

    void push_front(int x){
        ensure_full();
        size_t idx = prev_index(head);
        arr[idx] = x;
        head = idx;
        if (count < cap) count++;
    }

    void push_back(int x){
        ensure_full();
        if(cap==count){
            arr[this->head] = x;
            this->head = next_index(this->head);
        } else {
            arr[free_back_index()] = x;
            count++;
        }
    }

    int pop_front(){
        ensure_not_empty();
        int x = arr[head];
        head = next_index(head);
        count--;
        return x;
    }

    int pop_back(){
        ensure_not_empty();
        size_t idx = back_index();
        int x = arr[idx];
        count--;
        return x;
    }

    int& front(){
        ensure_not_empty();
        return arr[head];
    }
    int& back(){
        ensure_not_empty();
        return arr[back_index()];
    }

    int size() const{
        return count;
    }

    bool empty() const{
        return count == 0;
    }

    bool full() const{
        return count==cap;
    }

    void resize(size_t new_cap){
```

```
        if (new_cap == 0) throw invalid_argument("new_cap must be > 0");

        size_t new_count = (count < new_cap) ? count : new_cap;
        vector<int> new_arr(new_cap);
        for (size_t i = 0; i < new_count; i++) {
            new_arr[i] = arr[(head + i) % cap];
        }

        arr = new_arr;
        cap = new_cap;
        head = 0;
        count = new_count;
    }
};
```

Файл src/tests.cpp

```
#include <gtest/gtest.h>
#include <stdexcept>

// Класс CircularDeque вынесен в modules/circular_deque.h
#include "modules/circular_deque.h"

TEST(CircularDeque, CreateEmptyDeque) {
    CircularDeque dq(5, false);
    EXPECT_EQ(dq.size(), 0);
    EXPECT_TRUE(dq.empty());
    EXPECT_FALSE(dq.full());
}

TEST(CircularDeque, InvalidCapacityRaises) {
    EXPECT_THROW(CircularDeque(0, false), std::invalid_argument);
}

TEST(CircularDeque, FillDequeFromBothEnds) {
    CircularDeque dq(5, false);
    dq.push_front(1);
    dq.push_front(0);
    dq.push_back(2);
    dq.push_back(3);
    dq.push_back(4);

    EXPECT_EQ(dq.front(), 0);
    EXPECT_EQ(dq.back(), 4);
    EXPECT_TRUE(dq.full());
    EXPECT_EQ(dq.size(), 5);
}

TEST(CircularDeque, PushOnFullWithoutForceRaises) {
    CircularDeque dq(5, false);
    dq.push_front(1);
    dq.push_front(0);
    dq.push_back(2);
    dq.push_back(3);
    dq.push_back(4);

    EXPECT_THROW(dq.push_back(5), std::overflow_error);
    EXPECT_THROW(dq.push_front(5), std::overflow_error);
    EXPECT_EQ(dq.size(), 5);
    EXPECT_EQ(dq.front(), 0);
    EXPECT_EQ(dq.back(), 4);
}

TEST(CircularDeque, PopFromBothEnds) {
    CircularDeque dq(5, false);
    dq.push_front(1);
    dq.push_front(0);
    dq.push_back(2);
    dq.push_back(3);
    dq.push_back(4);
```

```
    EXPECT_EQ(dq.pop_back(), 4);
    EXPECT_EQ(dq.pop_back(), 3);
    EXPECT_EQ(dq.pop_back(), 2);
    EXPECT_EQ(dq.pop_front(), 0);
    EXPECT_EQ(dq.pop_back(), 1);
    EXPECT_TRUE(dq.empty());
}

TEST(CircularDeque, PopFromEmptyRaises) {
    CircularDeque dq(5, false);
    EXPECT_THROW(dq.pop_back(), std::underflow_error);
    EXPECT_THROW(dq.pop_front(), std::underflow_error);
}

TEST(CircularDeque, FrontBackOnEmptyRaise) {
    CircularDeque dq(1, false);
    EXPECT_THROW(dq.front(), std::underflow_error);
    EXPECT_THROW(dq.back(), std::underflow_error);
}

TEST(CircularDeque, PushForceEvictsOppositeEnd) {
    CircularDeque dq(3, true);
    dq.push_back(0);
    dq.push_back(1);
    dq.push_back(2);
    dq.push_front(3);
    dq.push_back(4);

    EXPECT_EQ(dq.front(), 0);
    EXPECT_EQ(dq.back(), 4);
    EXPECT_EQ(dq.size(), 3);
}

TEST(CircularDeque, PushForceExampleFromTask) {
    CircularDeque dq(3, true);
    dq.push_back(1);
    dq.push_back(2);
    dq.push_back(3);

    dq.push_back(4);

    EXPECT_EQ(dq.front(), 2);
    EXPECT_EQ(dq.back(), 4);
    EXPECT_EQ(dq.size(), 3);
}

TEST(CircularDeque, PushFrontForceEvictsBack) {
    CircularDeque dq(3, true);
    dq.push_back(1);
    dq.push_back(2);
    dq.push_back(3);

    dq.push_front(0);

    EXPECT_EQ(dq.front(), 0);
    EXPECT_EQ(dq.back(), 2);
    EXPECT_EQ(dq.size(), 3);
}

TEST(CircularDeque, ResizeGrowPreservesData) {
    CircularDeque dq(2, false);
    dq.push_back(1);
    dq.push_back(2);
    dq.resize(4);

    EXPECT_EQ(dq.size(), 2);
    EXPECT_EQ(dq.front(), 1);
    EXPECT_EQ(dq.back(), 2);
    EXPECT_FALSE(dq.full());
    dq.push_back(3);
    dq.push_front(0);
```

```
    EXPECT_EQ(dq.front(), 0);
    EXPECT_EQ(dq.back(), 3);
    EXPECT_TRUE(dq.full());
}

TEST(CircularDeque, ResizeShrinkTruncatesData) {
    CircularDeque dq(5, false);
    dq.push_back(1);
    dq.push_back(2);
    dq.push_back(3);
    dq.push_back(4);
    dq.resize(2);

    EXPECT_EQ(dq.size(), 2);
    EXPECT_TRUE(dq.full());
    EXPECT_EQ(dq.front(), 1);
    EXPECT_EQ(dq.back(), 2);
}

TEST(CircularDeque, ResizePreservesWrappedData) {
    CircularDeque dq(3, false);
    dq.push_back(1);
    dq.push_back(2);
    dq.push_back(3);
    dq.pop_front();
    dq.push_back(4);
    dq.resize(5);

    EXPECT_EQ(dq.size(), 3);
    EXPECT_EQ(dq.pop_front(), 2);
    EXPECT_EQ(dq.pop_front(), 3);
    EXPECT_EQ(dq.pop_front(), 4);
    EXPECT_TRUE(dq.empty());
}

TEST(CircularDeque, ResizeInvalidCapacityRaises) {
    CircularDeque dq(3, false);
    EXPECT_THROW(dq.resize(0), std::invalid_argument);
}

TEST(CircularDeque, PipelineFromReport) {
    CircularDeque dq(4, true);

    dq.push_back(0);
    dq.push_back(1);
    dq.push_front(2);
    EXPECT_EQ(dq.pop_back(), 1);
    dq.push_back(3);
    dq.push_back(4);
    EXPECT_TRUE(dq.full());

    dq.push_back(5);
    EXPECT_EQ(dq.pop_front(), 0);

    EXPECT_EQ(dq.size(), 3);
    EXPECT_EQ(dq.front(), 3);
    EXPECT_EQ(dq.back(), 5);
}

TEST(CircularDeque, SingleElementEdgeCases) {
    CircularDeque dq(1, false);
    dq.push_back(42);

    EXPECT_EQ(dq.front(), 42);
    EXPECT_EQ(dq.back(), 42);
    EXPECT_TRUE(dq.full());

    EXPECT_EQ(dq.pop_front(), 42);
    EXPECT_TRUE(dq.empty());
}
```

```
TEST(CircularDeque, SingleElementForceOverwrite) {
    CircularDeque dq(1, true);
    dq.push_back(1);
    dq.push_back(2);
    EXPECT_EQ(dq.front(), 2);
    EXPECT_EQ(dq.back(), 2);
    EXPECT_EQ(dq.size(), 1);

    dq.push_front(3);
    EXPECT_EQ(dq.front(), 3);
    EXPECT_EQ(dq.back(), 3);
    EXPECT_EQ(dq.size(), 1);
}

TEST(CircularDeque, FrontBackReturnMutableReference) {
    CircularDeque dq(3, false);
    dq.push_back(1);
    dq.push_back(2);

    dq.front() = 10;
    dq.back() = 20;

    EXPECT_EQ(dq.front(), 10);
    EXPECT_EQ(dq.back(), 20);
}

TEST(CircularDeque, WrapAroundManyOperations) {
    CircularDeque dq(3, false);
    for (int i = 0; i < 100; ++i) {
        dq.push_back(i);
        EXPECT_EQ(dq.back(), i);
        EXPECT_EQ(dq.pop_front(), i);
        EXPECT_TRUE(dq.empty());
    }
}
```

Файл src/modules/benchmark.cpp

```
#include <algorithm>
#include <chrono>
#include <cstdio>
#include <vector>

#include "circular_deque.h"

using clk = std::chrono::steady_clock;

static double ns_per_op(long long total_ns, long long ops) {
    return static_cast<double>(total_ns) / static_cast<double>(ops);
}

static long long median(std::vector<long long>& v) {
    std::sort(v.begin(), v.end());
    return v[v.size() / 2];
}

int main() {
    std::vector<size_t> sizes;
    for (size_t n = 1000; n <= 10000000; n = static_cast<size_t>(n * 1.5)) {
        sizes.push_back(n);
    }

    const int WARMUP = 1;
    const int REPEAT = 9;

    printf("operation,N,ns_per_op\n");

    for (size_t N : sizes) {
        std::vector<long long> samples;

        samples.clear();
```

```
        for (int r = 0; r < WARMUP + REPEAT; ++r) {
            CircularDeque dq(N, false);
            auto t0 = clk::now();
            for (size_t i = 0; i < N; ++i) dq.push_back(static_cast<int>(i));
            auto t1 = clk::now();
            long long dt =
                std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
t0).count();
            if (r >= WARMUP) samples.push_back(dt);
        }
        printf("push_back,%zu,%.4f\n", N, ns_per_op(median(samples), N));

        samples.clear();
        for (int r = 0; r < WARMUP + REPEAT; ++r) {
            CircularDeque dq(N, false);
            auto t0 = clk::now();
            for (size_t i = 0; i < N; ++i) dq.push_front(static_cast<int>(i));
            auto t1 = clk::now();
            long long dt =
                std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
t0).count();
            if (r >= WARMUP) samples.push_back(dt);
        }
        printf("push_front,%zu,%.4f\n", N, ns_per_op(median(samples), N));

        samples.clear();
        for (int r = 0; r < WARMUP + REPEAT; ++r) {
            CircularDeque dq(N, false);
            for (size_t i = 0; i < N; ++i) dq.push_back(static_cast<int>(i));
            auto t0 = clk::now();
            for (size_t i = 0; i < N; ++i) dq.pop_back();
            auto t1 = clk::now();
            long long dt =
                std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
t0).count();
            if (r >= WARMUP) samples.push_back(dt);
        }
        printf("pop_back,%zu,%.4f\n", N, ns_per_op(median(samples), N));

        samples.clear();
        for (int r = 0; r < WARMUP + REPEAT; ++r) {
            CircularDeque dq(N, false);
            for (size_t i = 0; i < N; ++i) dq.push_back(static_cast<int>(i));
            auto t0 = clk::now();
            for (size_t i = 0; i < N; ++i) dq.pop_front();
            auto t1 = clk::now();
            long long dt =
                std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
t0).count();
            if (r >= WARMUP) samples.push_back(dt);
        }
        printf("pop_front,%zu,%.4f\n", N, ns_per_op(median(samples), N));

        samples.clear();
        {
            volatile int sink = 0;
            for (int r = 0; r < WARMUP + REPEAT; ++r) {
                CircularDeque dq(N, false);
                for (size_t i = 0; i < N; ++i) dq.push_back(static_cast<int>(i));
                auto t0 = clk::now();
                for (size_t i = 0; i < N; ++i) sink = dq.front();
                auto t1 = clk::now();
                long long dt =
                    std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
t0).count();
                if (r >= WARMUP) samples.push_back(dt);
            }
            (void)sink;
        }
        printf("front,%zu,%.4f\n", N, ns_per_op(median(samples), N));
```

```
        samples.clear();
        {
            volatile int sink = 0;
            for (int r = 0; r < WARMUP + REPEAT; ++r) {
                CircularDeque dq(N, false);
                for (size_t i = 0; i < N; ++i) dq.push_back(static_cast<int>(i));
                auto t0 = clk::now();
                for (size_t i = 0; i < N; ++i) sink = dq.back();
                auto t1 = clk::now();
                long long dt =
                    std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
t0).count();
                if (r >= WARMUP) samples.push_back(dt);
            }
            (void)sink;
        }
        printf("back,%zu,%.4f\n", N, ns_per_op(median(samples), N));

        samples.clear();
        for (int r = 0; r < WARMUP + REPEAT; ++r) {
            CircularDeque dq(N, false);
            for (size_t i = 0; i < N; ++i) dq.push_back(static_cast<int>(i));
            auto t0 = clk::now();
            dq.resize(N * 2);
            auto t1 = clk::now();
            long long dt =
                std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
t0).count();
            if (r >= WARMUP) samples.push_back(dt);
        }

        printf("resize,%zu,%.4f\n", N, ns_per_op(median(samples), 1));
    }

    return 0;
}
```

Файл src/modules/plot_benchmark.py

```
import csv
import os
from collections import defaultdict

import numpy as np
import matplotlib.pyplot as plt

ROOT_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
CSV_PATH = os.path.join(ROOT_DIR, "benchmark.csv")
OUT_DIR = os.path.join(ROOT_DIR, "plots")

BLUE = "#2563eb"
RED = "#dc2626"

plt.rcParams.update({
    "figure.facecolor": "white",
    "axes.facecolor": "white",
    "font.size": 11,
})

THEORY = {
    "push_back": ("O(1)", lambda n: np.ones_like(n, dtype=float)),
    "push_front": ("O(1)", lambda n: np.ones_like(n, dtype=float)),
    "pop_back": ("O(1)", lambda n: np.ones_like(n, dtype=float)),
    "pop_front": ("O(1)", lambda n: np.ones_like(n, dtype=float)),
    "front": ("O(1)", lambda n: np.ones_like(n, dtype=float)),
    "back": ("O(1)", lambda n: np.ones_like(n, dtype=float)),
    "resize": ("O(n)", lambda n: n.astype(float)),
}
def load_data(path):
    data = defaultdict(lambda: ([], []))
    with open(path, newline="") as f:
```

```
        reader = csv.DictReader(f)
        for row in reader:
            op = row["operation"]
            data[op][0].append(int(row["N"]))
            data[op][1].append(float(row["ns_per_op"]))
    return data

def fit_scale(theory_vals, measured):
    """Подбор коэффициента c: minimize ||c * theory - measured||."""
    theory_vals = np.asarray(theory_vals, dtype=float)
    measured = np.asarray(measured, dtype=float)
    denom = float(np.dot(theory_vals, theory_vals))
    if denom == 0:
        return 0.0
    return float(np.dot(theory_vals, measured)) / denom
def main():
    if not os.path.exists(CSV_PATH):
        raise SystemExit(f"Не найден {CSV_PATH}. Сначала запустите ./benchmark >
{CSV_PATH}")

    os.makedirs(OUT_DIR, exist_ok=True)
    data = load_data(CSV_PATH)

    for op, (ns, ys) in data.items():
        order = np.argsort(ns)
        n = np.array(ns)[order]
        y = np.array(ys)[order]

        label, model = THEORY.get(op, ("?", lambda x: np.ones_like(x,
dtype=float)))
        theory = model(n)
        c = fit_scale(theory, y)
        theory_scaled = c * theory

        fig, ax = plt.subplots(figsize=(8, 5))

        if op == "resize":
            ax.plot(n, y, "o-", color=BLUE, markersize=6, linewidth=2,
                    label="Эксперимент (замер)", zorder=3)
            ax.plot(n, theory_scaled, "--", color=RED, linewidth=2,
                    label=f"Теория {label}", zorder=2)
            ax.set_xscale("log")
            ax.set_yscale("log")
            ax.set_ylabel("Время одного resize, нс")
        else:
            ax.plot(n, y, "o-", color=BLUE, markersize=6, linewidth=2,
                    label="Эксперимент (замер)", zorder=3)
            mean_y = float(np.mean(y))
            ax.axhline(mean_y, color=RED, linestyle="--", linewidth=2,
                       label=f"Теория {label} ≈ {mean_y:.2f} нс", zorder=2)
            ax.set_ylim(bottom=0, top=max(y.max(), mean_y) * 1.35)
            ax.set_xlabel("Размер N (элементов)")
            ax.set_ylabel("Время на операцию, нс")

        ax.set_xlabel("Размер N (элементов)")
        ax.set_title(f"CircularDeque::{op} — сравнение с теорией {label}",
                     fontsize=13, fontweight="bold", pad=12)
        ax.grid(True, which="both", alpha=0.25, linestyle="-")
        ax.legend(frameon=True, fontsize=10, loc="best")
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.margins(x=0.02)

        fig.tight_layout()
        out = os.path.join(OUT_DIR, f"{op}.png")
        fig.savefig(out, dpi=130)
        plt.close(fig)
        print(f"Сохранён график: {out}")

    print(f"\nГотово. Графики в каталоге {OUT_DIR}")
```

```
if __name__ == "__main__":
    main()
```

ПРИЛОЖЕНИЕ Б

Результаты тестирования

Таблица 1 - Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	CircularDeque(5, false)	size() = 0, empty() = true	Проверка создания пустого дека
2.	push_front(1) push_front(0) push_back(2) push_back(3) push_back(4)	front() = 0, back() = 4, full() = true, size() = 5	Добавление элементов и проверка заполненности
3.	push_back(5)	Возбуждается std::overflow_error	Реакция на переполнение при push_force = false
4.	pop_back() pop_back() pop_back() pop_front() pop_back()	Возвращает 4, затем 3, затем 2, затем 0, затем 1; empty() = true	Извлечение элементов с обоих концов
5.	pop_back()	Возбуждается std::underflow_error	Реакция на извлечение из пустого дека
6.	CircularDeque(3, true) push_back(0) push_back(1) push_back(2) push_front(3) push_back(4)	front() = 0, back() = 4, size() = 3	Вытеснение старых элементов при push_force = true
7.	CircularDeque(2, false) push_back(1) push_back(2) resize(4)	capacity = 4, size() = 2, front() = 1, back() = 2	Расширение буфера с сохранением данных
8.	resize(1)	size() = 1, front() = 1	Сжатие буфера: остаются первые элементы
9.	CircularDeque(1, false) back()	Возбуждается std::underflow_error	Чтение из пустого дека
10.	CircularDeque(0, false)	Возбуждается std::invalid_argument	Создание дека с нулевой вместимостью
11.	CircularDeque(3, true) push_back(1) push_back(2) push_back(3) push_back(4)	front() = 2, back() = 4, size() = 3	Пример из условия: push_back в полный дек вытесняет начало

12.	CircularDeque(3, true) push_back(1) push_back(2) push_back(3) push_front(0)	front() = 0, back() = 2, size() = 3	push_front в полный дек вытесняет конец
13.	CircularDeque(3, false) push_back(1) push_back(2) push_back(3) pop_front() push_back(4) resize(5)	size() = 3, pop_front() возвращает 2, затем 3, затем 4; empty() = true	resize сохраняет порядок, когда данные «переходят» через конец буфера
14.	CircularDeque(3, false) resize(0)	Возбуждается std::invalid_argument	Некорректная новая вместимость
15.	CircularDeque(4, true) push_back(0) push_back(1) push_front(2) pop_back() push_back(3) push_back(4) push_back(5) pop_front()	pop_back() = 1, full() = true перед push_back(5), pop_front() = 0, size() = 3, front() = 3, back() = 5	Pipeline данных из отчёта
16.	CircularDeque(1, false) push_back(42) pop_front()	front() = back() = 42, full() = true, pop_front() = 42, empty() = true	Граничный случай: дек из одного элемента

17.	CircularDeque(1, true)
	push_back(1)
	push_back(2)
	push_front(3)
		после push_back(2):
		front() = back() = 2;
		после push_front(3):
		front() = back() = 3;
		size() = 1
			Перезапись единственного элемента при push_force = true

18.	CircularDeque(3, false) push_back(1) push_back(2) front() = 10 back() = 20	front() = 10, back() = 20	front() и back() возвращают изменяемые ссылки
19.	CircularDeque(3, false) 100 раз: push_back(i), pop_front()	back() = i, pop_front() = i, empty() = true на каждой итерации	Многократный переход индексов через границу буфера